

Redes neurais artificiais

Prof. Dr. Oscar Eduardo Anacona Mosquera

oscar.mosquera@ufmt.br

30 de setembro de 2026

Sumário

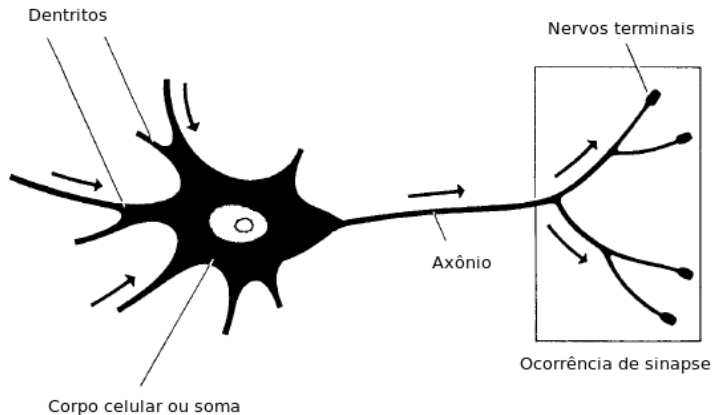
- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)

Sumário

- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)

Rede neural

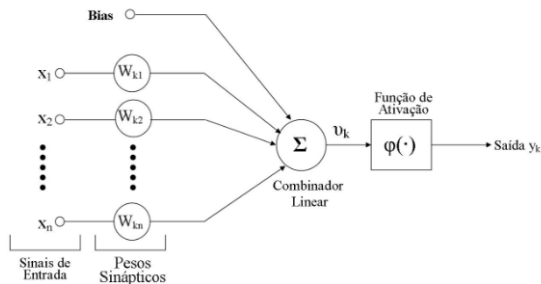
Um neurônio é a unidade básica do cérebro humano.



Fonte: Gonçalves

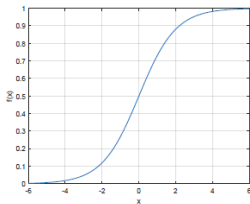
Rede Neural: modelo

Modelo de neurônio artificial proposto por McCulloch e Pitts. A soma é representada por uma composição de dois módulos: o primeiro é uma junção aditiva, somatório dos estímulos (sinais de entrada) multiplicado pelo seu fator excitatório (pesos sinápticos), e posteriormente uma função de ativação, que definirá, com base nas entradas e pesos sinápticos, qual será a saída do neurônio. O axônio é aqui representado pela saída (y_k) obtida pela aplicação da função de ativação.

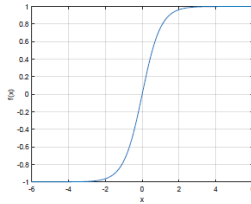


Fonte: Gonçalves

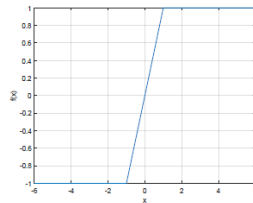
Rede Neural: funções de ativação



(a) Função Sigmoide $\frac{1}{1+e^{-x}}$



(b) Função Tanh $\frac{e^x - e^{-x}}{e^x + e^{-x}}$



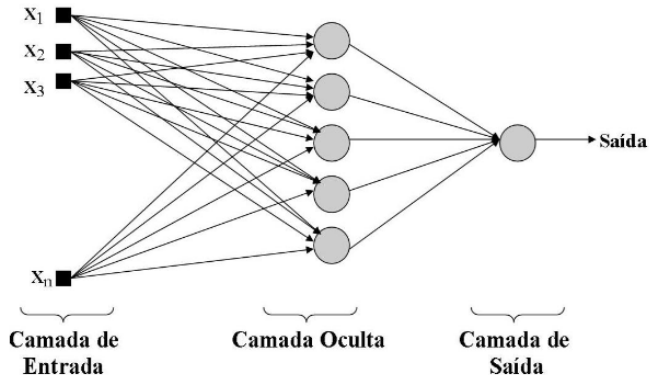
(c) Função Linear
 $FL = -1$ se $x \leq -1$, $FL =$
 x se $x > -1$ e $x < 1$, $FL =$
 1 se $x \geq 1$

Fonte:

TRIANA (2022)

Redes Multilayer Perceptron (MLP)

As redes MLP são redes do tipo *feedforward* (alimentadas adiante) que possuem uma ou mais camadas de neurônios entre as camadas de entrada e saída, denominadas camadas ocultas.



Fonte: Gonçalves

O algoritmo *backpropagation* utiliza a técnica de busca por gradiente para minimizar a função de custo.

Redes Multilayer Perceptron (MLP)

Algoritmo 1: Algoritmo de treinamento *backpropagation*

```
1 begin
2   Atribuição de valores iniciais aos pesos sinápticos;
3   repeat
4     Apresentação à rede dos padrões de entrada e as saídas desejadas;
5     Cálculo dos valores de saída dos neurônios ocultos;
6     Cálculo dos valores de saída dos neurônios de saída (resposta real da rede);
7     Cálculo do erro (diferença entre resposta da rede e valor esperado);
8     Ajuste dos pesos sinápticos;
9   until Condição de parada não satisfeita ;
10 end
```

Fonte: Gonçalves

Redes Multilayer Perceptron (MLP)

Algumas soluções para contornar limitações do gradiente (como a convergência para mínimos locais) foram sugeridas, como a aplicação de outras técnicas de aprendizado de máquina conhecidas pela alta aplicabilidade em problemas de otimização global, tais como:

- Algoritmos Genéticos;
- Otimização por Colônia de Formigas (*Ant Colony Optimization*).
- Otimização por enxame de partículas (PSO).
- Artificial Bee Colony (ABC).

Como a Rede Aprende? (Backpropagation)

O algoritmo de **Retropropagação** é o método usado para treinar a rede neural. Ele funciona em um ciclo de "tentativa e erro":

- ➊ **Passo à Frente (Forward):** A rede recebe os dados, faz os cálculos com os pesos atuais e gera um resultado.
- ➋ **Cálculo do Erro:** O computador compara o resultado da rede com a resposta correta e vê o tamanho do "vacilo"(erro).
- ➌ **Retropropagação (Backward):** O erro volta do final para o começo da rede, ajustando os **pesos** de cada neurônio.
- ➍ **Ajuste Fino:** O objetivo é diminuir o erro a cada rodada (época) até que a rede acerte quase tudo.

Por que treinar no Python?

Treinar exige muito cálculo matemático pesado. Fazemos o treinamento no PC (Python) e passamos apenas os **pesos prontos** para o FPGA.

Aplicações das Redes Neurais

Classificação de Padrões: Consiste em classificar padrões de entrada em classes previamente conhecidas, tomando como base de conhecimento as características intrínsecas do problema.

Clusterização: Pode ser vista como um problema de classificação de padrões onde não se conhece previamente a quantidade ou a natureza das classes. A rede agrupa os dados por similaridade.

Aplicações das Redes Neurais

Aproximação de Funções: Utilizada para modelar funções complexas e de alta não linearidade. Quando algoritmos exatos falham, as redes neurais surgem como uma alternativa robusta para aproximação.

Previsão: Com base em um histórico de cenários e nas ações tomadas anteriormente, a rede é capaz de prever qual será a resposta ou o valor esperado em um novo cenário futuro.

Aplicações das Redes Neurais

Otimização: Consiste em encontrar o valor máximo ou mínimo de uma função objetivo, sendo muito utilizada em problemas de logística e engenharia.

Associação: Desenvolvimento de redes que associam padrões. Pode ser treinada com dados ruidosos (*noise*) para identificar e recuperar o padrão original sem ruído.

Controle: Aplicação de redes neurais para auxiliar sistemas de controle adaptativo, ajustando parâmetros em tempo real para manter a estabilidade de um processo.

Redes de Função de Base Radial (RBF)

Dentre as arquiteturas utilizadas para as aplicações citadas, as redes **RBF** se destacam pela sua simplicidade estrutural e rapidez de treinamento.

- **Conceito:** Diferente das MLPs que utilizam funções sigmóides globais, as RBFs utilizam funções de ativação locais (geralmente Gaussianas).
- **Estrutura:** Composta por apenas três camadas:
 - 1 **Entrada:** Apenas distribui os dados.
 - 2 **Oculto (Radial):** Calcula a distância entre a entrada e um centro pré-definido.
 - 3 **Saída (Linear):** Realiza uma combinação linear dos resultados.

Motivação para Implementação em Hardware

Para aplicações de tempo real (como controle de sistemas industriais ou processamento de sinais de alta velocidade), a implementação em software nem sempre é eficiente.

Vantagens do Hardware Dedicado (FPGA/ASIC)

- **Paralelismo:** Diferente de um processador comum que executa instruções em série, o hardware pode calcular todos os neurônios simultaneamente.
- **Baixa Latência:** Resposta quase instantânea aos estímulos de entrada.
- **Eficiência Energética:** Circuitos otimizados para uma única tarefa.

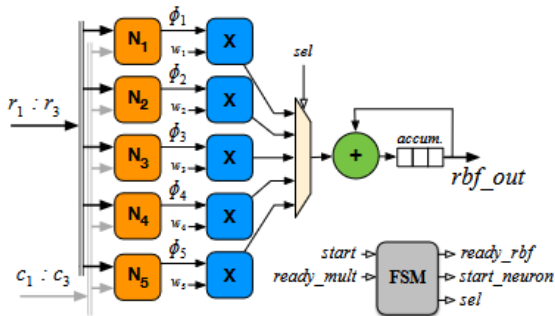
A seguir, veremos como essa estrutura matemática é mapeada em blocos digitais.

Arquitetura de Hardware RBF: Blocos Funcionais

A implementação digital da rede RBF é projetada para equilibrar paralelismo e consumo de área em hardware (FPGA/ASIC):

Camada de Processamento:

- **Blocos N_1 a N_5 :** Instâncias dos neurônios da camada oculta.
- Calculam a função de base radial de forma paralela.
- Entradas (r) e centros (c) são roteados simultaneamente para os blocos.



Fonte: Sampaio (2018)

Arquitetura de Hardware RBF: Fluxo de Dados

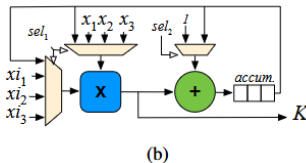
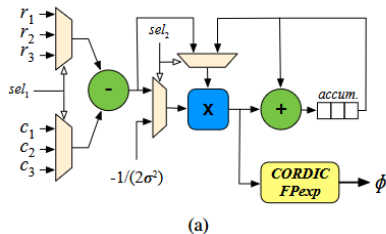
Para converter as saídas dos neurônios no resultado final (*rbf_out*), utiliza-se uma estrutura de **Multiplexação e Acúmulo**:

- **Multiplicadores ($\times$):** Cada saída ϕ_i é ponderada pelo seu peso w_i correspondente.
- **Multiplexador (MUX):** Seleciona sequencialmente cada produto para entrar no somador.
- **Somador/Acumulador:** Em vez de uma árvore de somadores (que gasta muita área), utiliza-se um registrador de acúmulo (*accum*) para somar os termos um a um.

Otimização de Hardware: Kernels

Um dos maiores desafios em hardware é a implementação da função Gaussiana devido à exponencial ($\exp$). Uma alternativa comum é o uso de **aproximações polinomiais**:

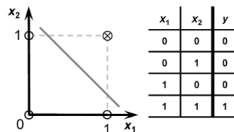
- **Kernel Gaussiano:** Alta precisão biológica/matemática, porém exige mais recursos (Elementos Lógicos) para implementar a exponencial.
- **Kernel Polinomial:** Exige significativamente menos hardware, sendo implementado apenas com multiplicadores e somadores simples.



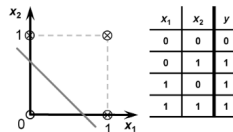
Fonte: Sampaio (2018)

Classificação de Padrões: Problemas Lineares

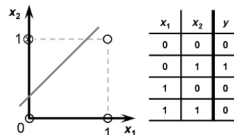
Muitas operações lógicas, como as portas **AND** e **OR**, são classificadas como problemas **linearmente separáveis**. Nesses casos, uma única reta (hiperplano) é capaz de dividir as classes no espaço de estados.



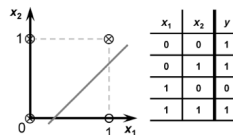
(a) $y \leftarrow (x_1) \text{ AND } (x_2)$



(b) $y \leftarrow (x_1) \text{ OR } (x_2)$



(c) $y \leftarrow (\text{NOT } x_1) \text{ AND } (x_2)$



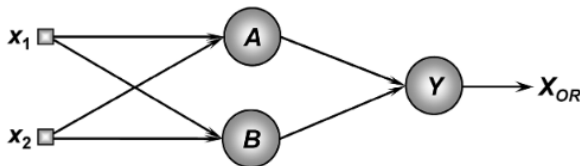
(d) $y \leftarrow (\text{NOT } x_1) \text{ OR } (x_2)$

Exemplos de operações lógicas linearmente separáveis.

Fonte: Silva, Spatti e Flauzino (2010).

Classificação de Padrões: Modelo de Neurônio

Para resolver problemas lineares, utiliza-se a arquitetura de um neurônio artificial simples (Perceptron), composto por duas entradas (x_1 , x_2) e uma saída (y).

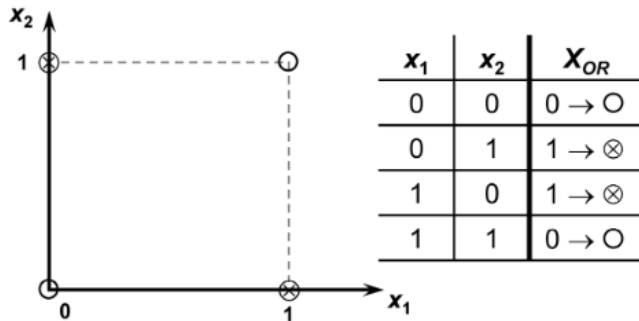


Representação de um neurônio com duas entradas e uma saída.

Fonte: Silva, Spatti e Flauzino (2010).

Classificação de Padrões: Problemas Não Lineares

Na operação **ou-exclusivo (XOR)**, constata-se que seria impossível posicionar uma única reta que permitisse separar as duas classes resultantes. Este é um exemplo clássico de um problema **não linearmente separável**.

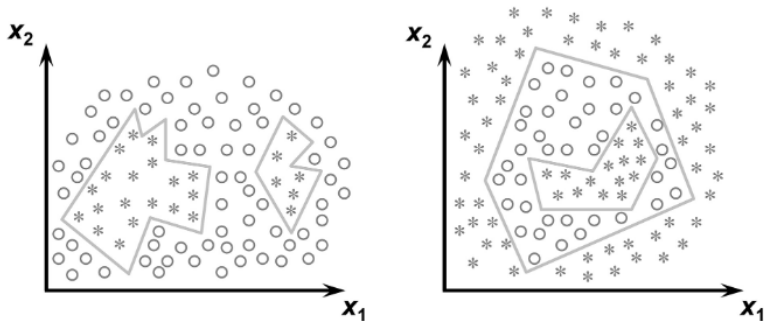


Distribuição espacial das classes da porta XOR.

Fonte: Silva, Spatti e Flauzino (2010).

Classificação de Padrões: Regiões Disjuntas

Para solucionar problemas de maior complexidade e com regiões disjuntas, é necessário o uso de múltiplas camadas ou funções de base radial, permitindo criar fronteiras de decisão complexas.



Exemplos de regiões disjuntas em classificação de padrões.

Fonte: Silva, Spatti e Flauzino (2010).

Sumário

- 1 Introdução
- 2 Ponto Flutuante**
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)

Representação Numérica: Ponto Flutuante (IEEE-754)

Para garantir a precisão dos cálculos neurais, utilizou-se o padrão **IEEE-754** de 32 bits. Contudo, em hardware, há um compromisso direto entre precisão e consumo de recursos.

- **Sinal (1 bit):** Define se o número é positivo ou negativo.
- **Expoente (8 bits):** Define a magnitude do número.
- **Mantissa (23 bits):** Define a precisão decimal.

Otimização de Recursos

Em FPGAs, multiplicadores de ponto flutuante consomem muitos *DSP Blocks*. Uma técnica explorada foi a **redução da mantissa** para observar a economia de elementos lógicos frente ao erro de aproximação.

Conversão Manual: Exemplo com o número 6,5

Para representar o número $6,5_{10}$ no padrão IEEE-754 de 32 bits:

- 1 **Binário:** $6_{10} = 110$ e $0,5_{10} = 0,1 \Rightarrow 110,1_2$
- 2 **Normalização:** $1,101 \times 2^2$ (movemos a vírgula 2 casas).
- 3 **Sinal (S):** 0 (positivo).
- 4 **Expoente (E):** $E = \text{deslocamento} + 2 = 127 + 2 = 129_{10}$
Em binário: 10000001_2
- 5 **Mantissa (M):** Parte fracionária de $1,101$ (omitindo o "1," implícito).
Resultado (23 bits): $10100000000000000000000_2$

Representação Final (Hexadecimal)

0 | 10000001 | 101000000000000000000000

Binário: 0100 0000 1101 0000 0000 0000 0000 0000 $\Rightarrow$ **0x40D00000**

Sumário

- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa**
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)

Experimento 1: Precisão vs. Largura da Mantissa

O impacto do truncamento vai ser avaliado configurando a mantissa em quatro resoluções distintas (11, 15, 19 e 23 bits), mantendo o expoente fixo em 8 bits (padrão IEEE-754).

- **Cenários de Teste:**

- ▶ **20-bits (M=11):** Alta economia de área; erro perceptível na 2ª casa decimal.
- ▶ **24-bits (M=15):** Equilíbrio inicial; erro na 3ª casa decimal.
- ▶ **28-bits (M=19):** Precisão próxima ao padrão comercial; erro apenas na 4ª casa.
- ▶ **32-bits (M=23):** Referência de precisão total (IEEE-754).

- **Observação no ModelSim:** O tempo de processamento (ciclos de clock) permanece constante, mas o valor final de y_{out} diverge conforme a mantissa diminui.

Redução de Mantissa e Simulação em VHDL

Para reduzir o consumo de hardware, podemos truncar a mantissa de 23 para 11 bits. Contudo, para manter a compatibilidade com o **ModelSim** (que espera 32 bits), preenchemos o restante com zeros (*zero padding*).

Código VHDL: Truncamento vs. Padding

```
-- Representacao de 6.5 com mantissa completa (23 bits)
constant FP_32 : std_logic_vector(31 downto 0) := x"40D00000";

-- Representacao com mantissa reduzida para 11 bits (Efeito de hardware)
-- Sinal(1) + Exp(8) + Mantissa_Reduzida(11) + Padding(12 zeros)
constant FP_RED : std_logic_vector(31 downto 0) :=
"0" & "10000001" & "101000000000" & "0000000000000";
```

- **No FPGA:** O circuito real utilizaria apenas os 20 bits superiores (1+8+11), economizando trilhas e lógica.
- **No ModelSim:** O *padding* permite que o simulador interprete o número como um decimal válido, permitindo verificar a perda de precisão da aproximação.

Sumário

- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global**
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)

Experimento 2: Treinamento e Otimização Global

O treinamento da rede MLP (arquitetura 2-2-1) foi realizado em Python utilizando a biblioteca **PyTorch**, focando na busca de pesos que garantam a convergência em hardware.

- **Arquitetura** (`nn.Module`): Duas camadas lineares (`Linear`) com funções de ativação Sigmóide.
- **Otimizador LBFGS**: Escolhido por sua alta eficiência em problemas de pequena escala, garantindo que a rede encontre o mínimo global ($\text{Loss} < 0.01$).
- **Estratégia de Reinicialização**: Implementação de um laço de busca automática que reinicia o treinamento caso a rede fique presa em um mínimo local (comum em problemas como XOR/OR).
- **Função de Perda**: `BCELoss` (Binary Cross Entropy), ideal para problemas de classificação binária.

Experimento 2: Conversão de Pesos para VHDL

Após a convergência, os pesos e *biases* (tensores) são extraídos e processados por uma função de conversão customizada para o padrão IEEE-754.

Lógica de Conversão (`float_to_bin_custom`)

- ❶ **Serialização:** O valor *float* é convertido em um inteiro de 32 bits via `struct.pack/unpack`.
 - ❷ **Decomposição:** Extração do sinal (1 bit), expoente (8 bits) e mantissa (23 bits).
 - ❸ **Truncamento Parametrizável:** A função permite ajustar o tamanho da mantissa (ex: reduzindo de 23 para 18 bits) para testar a economia de área no FPGA.
- **Saída Automática:** O script gera strings formatadas em VHDL (`weight_tb <= "0100..."`), eliminando erros manuais de transcrição para o *Testbench*.

Sumário

- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos**
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)

Experimento 3: VHDL Packages e Gestão de Pesos

Para garantir a modularidade, os pesos e tipos de dados foram centralizados em um **VHDL Package**. Isso facilita a atualização dos parâmetros treinados sem alterar a arquitetura dos neurônios.

- **Definição de Tipos:** Criação de *subtypes* para padronizar a largura de banda (ex: 32 bits).
- **Constantes Globais:** Armazenamento dos pesos e *bias* extraídos do PyTorch.

Exemplo de Estrutura do Package

```
package fpupack is
-- Definicao de tipo para facilitar a leitura
subtype fp32 is std_logic_vector(31 downto 0);

-- Pesos treinados (Exemplo da Camada Oculta)
constant C_W000 : fp32 := x"C18544DC"; -- Peso w00
constant C_W001 : fp32 := x"C165705A"; -- Peso w01
constant C_B00  : fp32 := x"4052262C"; -- Bias b00

-- Facilita a expansao para novos neuronios
end package;
```

Sumário

- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas**
- 7 Experimento 5: Integração e Simulação (ModelSim)

Experimento 4: Dinâmica das Operações Aritméticas

Diferente de somas inteiras simples, o ponto flutuante exige múltiplos passos internos (alinhamento de expoentes, normalização e tratamento de arredondamento).

- Como o tempo de processamento pode variar conforme os valores de entrada, o sistema precisa de sinais de coordenação:
 - ▶ **Sinal START:** O controlador principal "avisa" ao multiplicador ou somador que os dados de entrada são válidos e a operação deve começar.
 - ▶ **Sinal READY:** O módulo aritmético "responde" quando termina a normalização, garantindo que o dado na saída está correto e estável.
- **Sincronismo Assíncrono:** Essa técnica permite que o neurônio "espere" o tempo exato de cada componente, evitando que um dado seja lido antes de estar pronto (evitando erros de cálculo).

Fluxo de Execução do Neurônio

A operação fundamental do neurônio é processada como uma cadeia de eventos sequenciais coordenada por uma Máquina de Estados:

- ➊ **Etapa de Multiplicação:** O neurônio recebe o Peso (w) e a Entrada (x). O multiplicador é disparado. O sistema entra em estado de espera até que o sinal `ready_mul` suba.
- ➋ **Transferência Interna:** Assim que o produto ($w \cdot x$) está pronto, ele é automaticamente encaminhado para ser uma das entradas do somador.
- ➌ **Etapa de Soma (Acumulação):** O somador recebe o produto e o Bias (b). Um novo ciclo de processamento começa. O resultado final só é liberado para a função de ativação quando o `ready_as` confirma a estabilidade da soma.

Exemplo Numérico (IEEE-754)

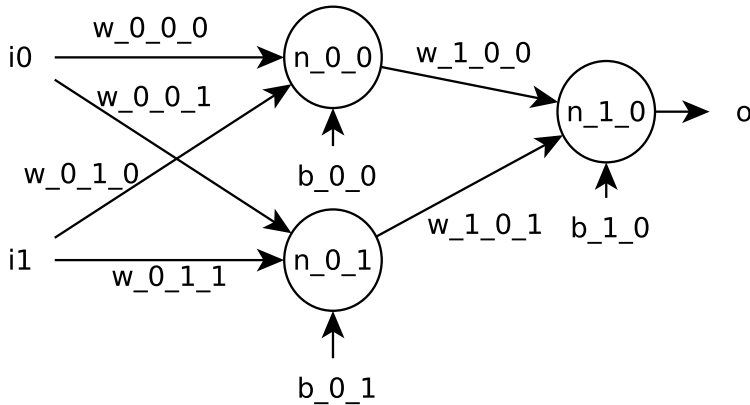
Para calcular $2.0 \times 1.5 + 0.5$:

- **Mult:** $0x40000000 \times 0x3FC00000 \rightarrow$ Gera $0x40400000$ (3.0).
- **Soma:** $3.0 + 0.5 \rightarrow$ Gera $0x40600000$ (3.5).

Sumário

- 1 Introdução
- 2 Ponto Flutuante
- 3 Experimento 1: Precisão vs. Largura da Mantissa
- 4 Experimento 2: Treinamento e Otimização Global
- 5 Experimento 3: VHDL Packages e Gestão de Pesos
- 6 Experimento 4: Dinâmica das Operações Aritméticas
- 7 Experimento 5: Integração e Simulação (ModelSim)**

Experimento 5: Relatório de Síntese



Experimento 5: Integração e Simulação (ModelSim)

A rede neural para a porta AND foi integrada em uma estrutura hierárquica e submetida a dois tipos de validação antes da gravação no FPGA:

- **Simulação Comportamental:** Validou-se a funcionalidade matemática. A FSM percorreu todos os estados e gerou os valores de saída esperados conforme o treinamento em PyTorch.
- **Simulação de Timing (Gate-Level):** Utilizou-se a *Netlist* gerada pelo Quartus para o chip da DE0-Nano.
 - ▶ **Resultado:** Confirmou-se que as operações de ponto flutuante estabilizam dentro do período de clock de 50MHz, sem violações de *setup* ou *hold*.
- **Verificação:** O sinal Ready comportou-se de forma estável, permitindo a leitura correta do resultado final após $\approx X$ ciclos de clock.

Experimento 5: Mapeamento Físico e Hardware

Para a implementação na placa DE0-Nano, foi desenvolvido um componente **Top Wrapper** para converter sinais digitais simples em dados de ponto flutuante:

- **Entradas (Switches):** As chaves físicas (SW0 e SW1) foram mapeadas para injetar constantes IEEE-754 (0.0 ou 1.0) na rede.
- **Controle (Push-Button):** Um botão físico foi configurado para disparar o sinal Start, iniciando o processamento da FSM.
- **Saída (LEDs):**
 - ▶ Um comparador de magnitude foi implementado para acender o LED apenas se a saída da rede for > 0.5 .
 - ▶ Um segundo LED foi utilizado para sinalizar o estado Ready.

Experimento 5: Validação da Porta AND

A tabela abaixo apresenta os resultados obtidos após a programação da placa, comparando a saída teórica com os valores extraídos via Signal Tap e LEDs:

A	B	Teoria (AND)	Saída FP (Rede)	LED Status
0	0	0	0.0000...	APAGADO
0	1	0	0.0001...	APAGADO
1	0	0	0.0001...	APAGADO
1	1	1	0.9998...	ACESO

Conclusão

A rede neural convergiu com precisão elevada. A pequena variação decimal (0.9998 em vez de 1.0) é característica da função de ativação Sigmóide, sendo interpretada corretamente pelo hardware.

Experimento 5: Relatório de Síntese

A imagem abaixo detalha a ocupação da área lógica na Cyclone IV após a síntese final do projeto:

Flow Status	Successful - Wed Sep 30 14:56:22 2026
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Standard Edition
Revision Name	rede
Top-level Entity Name	Top_Wrapper
Family	Cyclone IV E
Device	EP4CE22F17C6
Timing Models	Final
Total logic elements	5,602 / 22,320 (25 %)
Total registers	1368
Total pins	7 / 154 (5 %)
Total virtual pins	0
Total memory bits	1,024 / 608,256 (< 1 %)
Embedded Multiplier 9-bit elements	28 / 132 (21 %)
Total PLLs	0 / 4 (0 %)

- **Eficiência:** O uso de blocos de DSP dedicados permitiu a implementação de multiplicadores de ponto flutuante sem sobrecarregar as LUTs (Logic Elements).
- **Escalabilidade:** O baixo consumo de recursos indica que a DE0-Nano suportaria redes significativamente maiores com a mesma arquitetura modular.

Referências Bibliográficas I

- SAMPAIO, Renato. **Arquiteturas de hardware para aceleração de algoritmos de controle preditivo não-linear**. 2018. 120 f. Tese (Doutorado em Engenharia Mecânica) — Universidade de Brasília, Brasília, 2018.
- TRIANA, Mario Andrés Pastrana. **Controle baseado em comportamento aplicado à robótica móvel usando hardware reconfigurável**. Orientador: Daniel Mauricio Muñoz Arboleda. 2022. 88 f. Dissertação (Mestrado em Sistemas Mecatrônicos) — Universidade de Brasília, Brasília, 2022.
- GONÇALVES, André Ricardo. **Redes Neurais Artificiais**. Departamento de Engenharia de Computação e Automação Industrial (DCA) — UNICAMP, Campinas, SP.
- SILVA, Ivan Nunes da; SPATTI, Danilo Hernane; FLAUZINO, Rogério Andrade. **Redes neurais artificiais para engenharia e ciências aplicadas: curso prático**. São Paulo: Artliber Editora, 2010. 399 p.